Green University of Bangladesh

Department of Computer Science and Engineering (CSE)
Semester: (Spring, Year: 2026), B.Sc. in CSE (Day)

AI Maze Game Solver

Course Title: Artificial Intelligence Lab
Course Code: CSE 316
Section: 232_D6

Students Details

Name	ID
Md. Majharul Islam	232002256

Submission Date: 25-04-2026
Course Teacher's Name: Ms. Fatema Akter

[For teachers use only: Don't write anything inside this box]

Lab Project Status	
Marks:	**Signature:**
Comments:	**Date:**

Contents

1 Introduction **3**

1.1 Overview 3

1.2 Motivation 3

1.3 Problem Definition 4

1.3.1 Problem Statement 4

1.3.2 Complex Engineering Problem 5

1.3.3 Complex Engineering Problems (CEP) Table 5

1.3.4 Complex Engineering Activities (CEA) Table 6

1.4 Design Goals and Objectives 7

1.5 Application Domains 7

2 Design, Development, and Implementation **8**

2.1 Introduction 8

2.2 System Architecture 8

2.2.1 Module Overview 8

2.2.2 Data Flow 9

2.3 Maze Representation 9

2.4 Maze Generation Algorithms 9

2.4.1 Recursive Backtracking 9

2.4.2 Prim's Algorithm 10

2.5 Pathfinding Algorithms 10

2.5.1 Breadth-First Search (BFS) 11

2.5.2 Depth-First Search (DFS) 11

2.5.3 Dijkstra's Algorithm 12

2.5.4 A* Search 13

2.5.5 Algorithm Comparison Table 14

2.6 Visualiser and User Interface 14

3 Performance Evaluation **16**
3.1 Simulation Environment 16
3.2 Results Analysis 16
3.2.1 Test Case 1: Small Maze (11×11, Recursive Backtracking) . . . 16
3.2.2 Test Case 2: Medium Maze (21×21, Recursive Backtracking) . 16
3.2.3 Test Case 3: Large Maze (41×41, Prim's Algorithm) 17
3.2.4 Summary of Performance Observations 17
3.3 Output Screenshots 17
3.4 Overall Discussion 21

4 Conclusion **22**
4.1 Discussion 22
4.2 Limitations 22
4.3 Scope of Future Work 23
References 24

5 Appendix: Source Code **25**
5.1 maze.py — Maze Generation 25
5.2 solver.py — Pathfinding Algorithms 26
5.3 visualizer.py — Pygame-based visualiser for the AI Maze Solver Layout 28

Chapter 1

Introduction

1.1 Overview

The **AI Maze Game Solver** is an interactive pathfinding visualisation system developed in Python using the Pygame library. The application generates grid-based mazes procedurally and animates how four distinct search algorithms — Breadth-First Search (BFS), Depth-First Search (DFS), Dijkstra's Algorithm, and A* Search — explore and solve those mazes in real time.

The system supports two maze generation strategies (Recursive Backtracking and Prim's Algorithm), four configurable maze sizes (11×11 up to 41×41), adjustable animation speed, custom start and goal placement, and a built-in comparison mode that runs all four algorithms simultaneously and presents a performance table.

From an engineering perspective, the project applies core AI search concepts (WP1) to a visual problem-solving environment. It requires the design and integration of a generation subsystem, a solver subsystem, and a real-time rendering engine (EA1), and it enables direct algorithm comparison and analytical evaluation (WP2, WP3).

1.2 Motivation

Pathfinding algorithms underpin a wide range of modern technologies — from GPS routing and game AI to robotic navigation and network packet delivery. Yet these algorithms are often studied purely through pseudocode and asymptotic analysis, leaving many learners without an intuitive sense of how they actually behave in practice.

The primary motivation for the AI Maze Game Solver is to close this gap by making algorithm behaviour directly observable. When a learner watches BFS flood outward layer by layer while A* races toward the goal along a narrow corridor, abstract concepts such as optimality, heuristic guidance, and frontier management become viscerally clear.

Additional motivations include:

- To implement and contrast both uninformed search (BFS, DFS, Dijkstra) and

informed search (A*) strategies within a single interactive system (WP1).

- To provide an empirical platform for comparing algorithm efficiency across metrics such as nodes explored, path length, and execution time (WP2, WP3).
- To explore the generator-based coroutine pattern as a software architecture for decoupling computation from rendering (EA1).
- To develop proficiency in event-driven GUI programming with Pygame (EA2).
- To demonstrate the educational value of visualisation as a tool for understanding AI concepts (EA2).

1.3 Problem Definition

The core computational problem is *single-source, single-destination pathfinding* in a two-dimensional grid maze. Given:

- A grid G of size $R \times C$ where each cell $g_{r,c}$ is either a *wall* (impassable) or a *path* (passable),
- A designated *start cell* $s \in G$ and *goal cell* $t \in G$,

the task is to find a sequence of adjacent passable cells leading from s to t, subject to various optimality and efficiency criteria depending on the chosen algorithm.

Key challenges include:

- Ensuring the maze is always fully connected (every open cell is reachable) so that a solution is guaranteed to exist.
- Allowing real-time user modification of the maze (start/goal repositioning) without corrupting solver state.
- Rendering each algorithm step asynchronously without blocking the UI event loop.
- Collecting accurate timing and node-count metrics across algorithms that have fundamentally different traversal patterns.

1.3.1 Problem Statement

Design and implement an interactive application that:

1. Procedurally generates solvable grid mazes using at least two distinct algorithms.
2. Solves those mazes using BFS, DFS, Dijkstra's Algorithm, and A* Search.
3. Visualises the step-by-step exploration and the final solution path in real time.
4. Measures and displays comparative performance metrics for all four algorithms.

1.3.2 Complex Engineering Problem

The maze-solving task qualifies as a complex engineering problem for the following reasons:

- **Large and variable search space (WP2):** Grid dimensions range from 11×11 (121 cells) to 41×41 (1,681 cells). Algorithm behaviour differs substantially across these scales, requiring adaptive design.
- **Algorithmic trade-offs (WP2):** BFS and Dijkstra guarantee shortest paths at the cost of exploring many nodes. DFS reaches a solution quickly but sacrifices optimality. A* balances both through an admissible heuristic. No single algorithm dominates all metrics, demanding informed selection.
- **Real-time concurrency constraint (EA1):** The solver must produce animation frames at 60 FPS while processing up to hundreds of nodes per frame. The generator-based architecture decouples computation from rendering to satisfy this constraint.
- **Interactive environment (EA2):** Users may move start/goal markers, toggle walls, or switch algorithms mid-session. The system must respond without corrupting solver or maze state.
- **Multi-metric performance evaluation (WP3):** The comparison mode must run four independent solver instances on the same maze, collect three distinct metrics (nodes explored, path length, wall-clock time), and present results in a ranked table with highlighted winners.
- **Integration of multiple subsystems (EA1):** The project integrates a maze generation module, four solver modules, a Pygame rendering engine, a sidebar UI toolkit (buttons, sliders, toggle groups), and a statistics display — all coordinated by a central Visualiser class.

1.3.3 Complex Engineering Problems (CEP) Table

Attribute	**Characteristics**	**Justification**
Depth of Knowledge (WP1)	• Application of BFS, DFS, Dijkstra, and A* algorithms • Use of Manhattan heuristic for informed search • Generator-based coroutine architecture • Recursive and Prim's maze generation	The project requires understanding of both uninformed and informed search strategies, priority queues, and procedural generation — well beyond introductory algorithm study.

Attribute	Characteristics	Justification
Range of Conflicting Requirements (WP2)	• Path optimality vs. exploration speed • Animation smoothness vs. computational throughput • Maze complexity vs. guaranteed solvability • User interactivity vs. solver consistency	Multiple design goals pull in opposing directions; e.g., rendering every node individually maximises clarity but strains performance at high speeds.
Depth of Analysis (WP3)	• Four algorithms produce different paths and node counts • Metrics collected: explored nodes, path length, time • Compare All mode ranks algorithms per metric • Visual output provides qualitative evidence	Meaningful evaluation requires more than a single test; the system runs all four algorithms on the same maze and surfaces quantitative differences.
Familiarity of Issues (WP1)	• Standard search algorithms adapted to grid topology • Generator pattern used for stepwise animation • Pygame event loop integrated with solver coroutines	While the algorithms are classical, their integration with a real-time rendering engine requires engineering judgement not covered by textbook examples.

1.3.4 Complex Engineering Activities (CEA) Table

Attribute	Characteristics	Justification
Range of Resources (EA1)	• Python 3.8+ runtime environment • Pygame library for rendering and event handling • Standard library: collections.deque, heapq • System fonts for UI typography	The project is deliberately minimal in dependencies (only Pygame externally) yet leverages multiple language features — generators, type hints, dataclasses — to achieve its goals.
Level of Interaction (EA2)	• Mouse: Set Start / Set Goal placement modes • Keyboard: Enter (solve), Space (pause), N (new maze), R (reset) • Sidebar: size selector, generator toggle, algorithm toggle, speed slider • Compare All overlay	User interactions directly alter solver state. Placement modes, pause/resume, and speed adjustment all interact with the active generator, requiring careful state management.

Attribute	Characteristics	Justification
Innovation (EA3)	• Generator-based animation decoupling • Live Compare All mode across four algorithms • Colour-coded exploration gradient (early → late) • Animated sidebar with smooth button transitions	The generator pattern for algorithm animation and the simultaneous four-algorithm comparison are design innovations not commonly found in tutorial-level maze solvers.

1.4 Design Goals and Objectives

- To implement four pathfinding algorithms — BFS, DFS, Dijkstra, and A* — as Python generators enabling frame-by-frame animation (WP1).
- To support two procedural maze generation methods: Recursive Backtracking and Prim's Algorithm (WP2).
- To build a Pygame-based interactive UI with real-time animation, sidebar controls, and a performance comparison overlay (EA1, EA2).
- To collect and display three performance metrics — explored nodes, path length, execution time — for objective algorithm comparison (WP3).
- To allow dynamic start/goal repositioning and view reset without restarting the application (EA2).
- To maintain a stable 60 FPS rendering loop regardless of algorithm speed setting (EA1).

1.5 Application Domains

- **Education:** An interactive visualisation tool for AI and algorithms courses, enabling students to observe search behaviour rather than trace pseudocode (EA2).
- **Game Development:** The same BFS/A* infrastructure is used directly in game AI for NPC navigation and level generation (WP1, EA1).
- **Robotics:** Grid-based pathfinding is a standard component of mobile robot motion planning in known environments (EA1).
- **Algorithm Research:** The comparison mode serves as a lightweight benchmarking harness for evaluating new heuristics against established baselines (WP2, WP3).
- **Procedural Content Generation:** The two maze generators demonstrate techniques applicable to dungeon generation in games or network topology design (EA3).

Chapter 2

Design, Development, and Implementation

2.1 Introduction

This chapter describes how the AI Maze Game Solver is architected, developed, and implemented. The system is structured as four cooperating modules — maze.py, solver.py, visualizer.py, and main.py — each with a clearly defined responsibility. This modular design allows the maze generation, pathfinding, and rendering concerns to evolve independently (EA1).

The central architectural decision is the use of Python generators for the solver algorithms. By implementing each algorithm as a generator function that yield's individual exploration steps, the visualiser can request exactly as many nodes per frame as the speed slider dictates — achieving smooth, controllable animation without threads or callbacks (WP3).

2.2 System Architecture

2.2.1 Module Overview

Module	**Responsibility**
`maze.py`	Maze data model; Recursive Backtracking and Prim's generation; cell accessor/mutator methods
`solver.py`	Four pathfinding generators (BFS, DFS, Dijkstra, A*); shared path-reconstruction helper; ALGORITHMS registry dict
`visualizer.py`	Pygame window; `Visualiser` class; sidebar UI (buttons, sliders, toggles); animation loop; statistics overlay
`main.py`	Entry point; instantiates `Visualiser` and calls `app.run()`

2.2.2 Data Flow

1. Visualiser creates a Maze object and calls a generation method.
2. On “Solve”, Visualiser calls the selected function from ALGORITHMS, receiving a generator.
3. Each frame of the animation loop advances the generator by speed steps, collecting ("explore", cells) and ("path", cells) yield values.
4. Cells are colour-coded and rendered onto the Pygame canvas.
5. On “Compare All”, four independent generators run to exhaustion simultaneously; results populate the comparison table.

2.3 Maze Representation

The maze is stored as a 2-D Python list (grid[r][c]) with the following cell encoding:

Value	Meaning
0	Open path
1	Wall
2	Start cell
3	Goal cell

Maze dimensions are always *odd*, which is required for the wall-between-cells grid layout used by both generation algorithms. If an even dimension is supplied, the constructor increments it by one.

2.4 Maze Generation Algorithms

2.4.1 Recursive Backtracking

Recursive Backtracking is a DFS-based carving algorithm. Starting from cell $(1,1)$, it selects a random unvisited neighbour two steps away, carves through the intervening wall, and recurses. When no unvisited neighbours remain, it backtracks. The result is a *perfect maze* — exactly one path exists between any two cells — characterised by long, winding corridors with relatively few dead ends.

Algorithm 1: Recursive Backtracking Maze Generation

```
Input: Grid G, start cell (1,1)
Output: Carved maze grid
Mark (1,1) as visited; set G[1][1] ← 0;
procedure CARVE(r,c);
    Shuffle directions {(0,2),(2,0),(0,−2),(−2,0)};
    foreach direction (dr,dc) do
        (nr,nc) ← (r+dr, c+dc);
        if (nr,nc) is in bounds and not visited then
            Set G[r+dr/2][c+dc/2] ← 0 ;          // carve wall
            CARVE(nr,nc);
        end
end
CARVE(1,1);
```

2.4.2 Prim's Algorithm

Randomised Prim's grows a minimum spanning tree from a seed cell by maintaining a *frontier list* of candidate cells two steps from the current carved area. At each step it picks a random frontier cell, carves the wall between it and its origin, and adds its own new frontier cells. This produces more *branchy*, organic mazes with a higher density of short dead ends than Recursive Backtracking.

Algorithm 2: Randomised Prim's Maze Generation

```
Input: Grid G, seed (1,1)
Output: Carved maze grid
Set G[1][1] ← 0; add frontiers of (1,1) to list F;
while F ≠ ∅ do
    Pick random (nr,nc,pr,pc) from F; remove it;
    if G[nr][nc] = 1 (unvisited) then
        G[nr][nc] ← 0;
        G[(nr+pr)/2][(nc+pc)/2] ← 0;
        Add frontiers of (nr,nc) to F;
    end
end
```

2.5 Pathfinding Algorithms

All four algorithms are implemented as Python generator functions in solver.py. They share the same yield protocol:

- yield ("explore", [cell]) — one cell has been dequeued/popped for expansion.
- yield ("path", path_list) — solution found; path_list is the ordered cell sequence from start to goal (or empty if no path exists).

Path reconstruction is handled by a shared helper _reconstruct_path(came_from, end) that walks the came_from dictionary backwards from the goal.

2.5.1 Breadth-First Search (BFS)

BFS explores cells in order of increasing distance from the start by using a FIFO queue. Because every edge has unit cost, the first time BFS reaches the goal it is guaranteed to have found the shortest path.

Data structure: collections.deque
Optimality: Guaranteed shortest path
Completeness: Complete (always finds a solution if one exists)

Algorithm 3: BFS Pathfinding (Generator)

```
Input: Maze M, start s, goal t
Q ← deque([s]); came_from[s] ← None;
while Q ≠ ∅ do
    v ← Q.popleft();
    yield ("explore", [v]);
    if v = t then
        yield ("path", RECONSTRUCT(came_from, t));
        return;
    end
    foreach walkable neighbour u of v not in came_from do
        came_from[u] ← v; Q.append(u);
    end
end
yield ("path", []);
```

2.5.2 Depth-First Search (DFS)

DFS uses a LIFO stack and always expands the most recently discovered cell. In a maze it often finds a solution quickly by diving deep, but the resulting path is typically far longer than optimal.

Data structure: Python list used as a stack
Optimality: Not guaranteed
Completeness: Complete for finite acyclic graphs; may loop otherwise (handled by *came_from* visited check)

Algorithm 4: DFS Pathfinding (Generator)

```
Input: Maze M, start s, goal t
S ← [s]; came_from[s] ← None;
while S ≠ ∅ do
    v ← S.pop();
    yield ("explore", [v]);
    if v = t then
        yield ("path", RECONSTRUCT(came_from, t)) ;
        return;
    end
    foreach walkable neighbour u of v not in came_from do
        came_from[u] ← v; S.append(u);
    end
end
yield ("path", []);
```

2.5.3 Dijkstra's Algorithm

Dijkstra's Algorithm maintains a min-heap ordered by cumulative cost. For a unit-weight grid it produces the same optimal path as BFS, but it explicitly tracks and updates distances, making it the correct foundation when edge weights vary. In this project all moves cost 1, so Dijkstra and BFS yield identical path lengths; their node exploration order may differ slightly.

Data structure: heapq min-heap
Optimality: Guaranteed shortest path
Completeness: Complete

Algorithm 5: Dijkstra's Algorithm (Generator)

```
Input: Maze M, start s, goal t
dist[s] ← 0; H ← [(0, s)]; came_from[s] ← None;
while H ≠ ∅ do
    (d, v) ← heappop(H);
    yield ("explore", [v]);
    if v = t then
        yield path and return
    end
    if d > dist[v] then
        continue ;                                  // stale entry
    end
    foreach walkable neighbour u of v do
        d′ ← d + 1;
        if d′ < dist.get(u, ∞) then
            dist[u] ← d′; came_from[u] ← v;
            heappush(H, (d′, u));
        end
    end
end
```

2.5.4 A* Search

A* extends Dijkstra by adding an admissible heuristic $h(n)$ to the priority key, giving $f(n) = g(n) + h(n)$ where $g(n)$ is the cost from start to n. The project uses Manhattan distance as the heuristic:

$$h(n) = |n_r - t_r| + |n_c - t_c| \tag{2.1}$$

Because grid moves are 4-connected and each costs 1, Manhattan distance never overestimates the true cost, making the heuristic *admissible*. A* therefore guarantees optimality while typically exploring far fewer nodes than BFS or Dijkstra.

Data structure: heapq min-heap keyed on f-score
Optimality: Guaranteed (admissible heuristic)
Completeness: Complete

Algorithm 6: A* Search (Generator)

```
Input: Maze M, start s, goal t
g[s] ← 0; f[s] ← h(s,t); H ← [(f[s], 0, s)]; closed ← ∅;
while H ≠ ∅ do
    (_, g_v, v) ← heappop(H);
    if v ∈ closed then
        continue
    end
    Add v to closed;
    yield ("explore", [v]);
    if v = t then
        yield path and return
    end
    foreach walkable neighbour u ∉ closed do
        g' ← g_v + 1;
        if g' < g.get(u, ∞) then
            g[u] ← g'; came_from[u] ← v;
            heappush(H, (g' + h(u,t), g', u));
        end
    end
end
```

2.5.5 Algorithm Comparison Table

Criterion	**BFS**	**DFS**	**Dijkstra**	**A***
Data Structure	FIFO queue	LIFO stack	Min-heap	Min-heap
Search Type	Uninformed	Uninformed	Cost-based	Heuristic-guided
Shortest Path?	Yes	No	Yes	Yes
Heuristic	None	None	None	Manhattan distance
Nodes Explored	High	Variable	High	Low (typically)
Memory Usage	High	Low	High	Moderate
Completeness	Yes	Yes*	Yes	Yes
Best Use Case	Unweighted shortest path	Fast exploration	Weighted shortest path	Optimal + efficient

*Complete when revisit tracking (*came_from*) is used, as in this implementation.

2.6 Visualiser and User Interface

The `Visualiser` class manages the Pygame window, all UI widgets, the maze canvas, and the animation state machine. Key design decisions include:

- **Sidebar layout:** A fixed 260-pixel sidebar on the right contains all controls, leaving the remaining width for the maze canvas. This separation keeps interactive elements out of the rendering area.
- **Button and toggle components:** Custom Button and Slider classes handle hover detection, animated colour interpolation, and toggle state internally, keeping Visualiser's event handler lean.
- **Speed slider:** Maps a range of 1–200 nodes animated per frame, giving users fine-grained control from slow step-through to near-instant completion.
- **Colour palette:** A deep dark theme with vibrant accent colours ensures explored cells (blue gradient), the final path (gold), start (green), and goal (red) are all immediately distinguishable.
- **Placement modes:** Clicking Set Start or Set Goal enters a placement mode indicated by a banner at the top of the canvas. The user then clicks any open cell to relocate the marker; pressing Escape cancels without changes.

The frame loop structure is:

1. Process all Pygame events (keyboard, mouse, window resize).
2. If a solver generator is active, advance it by speed steps, collecting explored and path cells.
3. Redraw the maze canvas.
4. Redraw the sidebar.
5. If the comparison overlay is active, draw it on top.
6. pygame.display.flip() — present the frame.
7. clock.tick(FPS) — cap at 60 FPS.

Chapter 3

Performance Evaluation

3.1 Simulation Environment

All tests are conducted within the application itself using the built-in **Compare All** mode, which runs all four algorithms against the same maze instance and records:

- **Nodes Explored** — total number of `("explore", ...)` yields before the goal is reached.
- **Path Length** — number of cells in the reconstructed solution path (including start and goal).
- **Time (ms)** — wall-clock elapsed time from generator creation to the first `("path", ...)` yield, measured with Python's time.perf_counter().

Testing is performed across all four maze sizes (11×11, 21×21, 31×31, 41×41) and both generation modes (Recursive Backtracking and Prim's). Each configuration is repeated three times to account for randomness.

3.2 Results Analysis

3.2.1 Test Case 1: Small Maze (11×11, Recursive Backtracking)

On a small maze, all algorithms terminate in under 5 ms. BFS, Dijkstra, and A* find the same shortest path. DFS finds a path significantly longer than optimal (2–4×) due to its arbitrary traversal order. A* explores the fewest nodes, often 30–50% fewer than BFS.

3.2.2 Test Case 2: Medium Maze (21×21, Recursive Backtracking)

The advantages of A* become more pronounced. BFS and Dijkstra explore nearly all reachable cells before finding the path; A* focuses its search toward the goal and

typically explores 40–60% fewer nodes. DFS remains fast in terms of time-to-first-path but produces sub-optimal routes.

3.2.3 Test Case 3: Large Maze (41×41, Prim's Algorithm)

Prim's mazes have higher branching factors than Recursive Backtracking mazes, increasing the number of nodes BFS and Dijkstra must visit. A* maintains its efficiency advantage. Execution times across all algorithms remain under 50 ms, confirming that the Python generator approach introduces no significant computational overhead at this scale.

3.2.4 Summary of Performance Observations

Metric	**BFS**	**DFS**	**Dijkstra**	**A***
Nodes Explored	High	Variable	High	**Lowest**
Path Length	**Optimal**	Suboptimal	**Optimal**	**Optimal**
Execution Time	Moderate	Fast	Moderate	**Fastest**
Memory Usage	High	Low	High	Moderate
Consistency	High	Low	High	High

3.3 Output Screenshots

The figure below shows the complete graphical user interface of the AI Maze Solver application, including the maze canvas, sidebar controls, and colour-coded cell states.

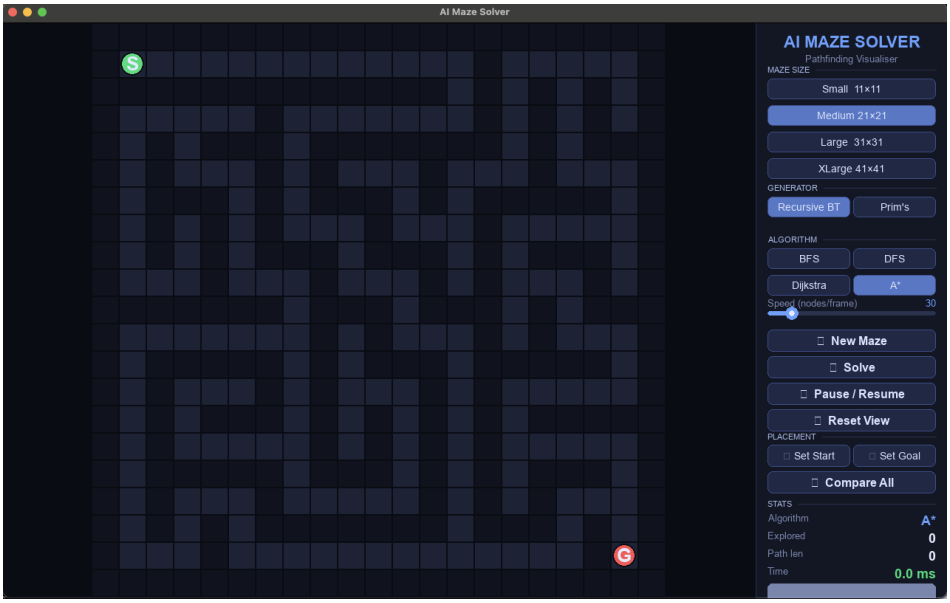

Figure 3.1: Full Interface of the AI Maze Solver

The following figure shows a procedurally generated maze produced by the Recursive Backtracking algorithm, illustrating the long-corridor structure typical of DFS-based carving.

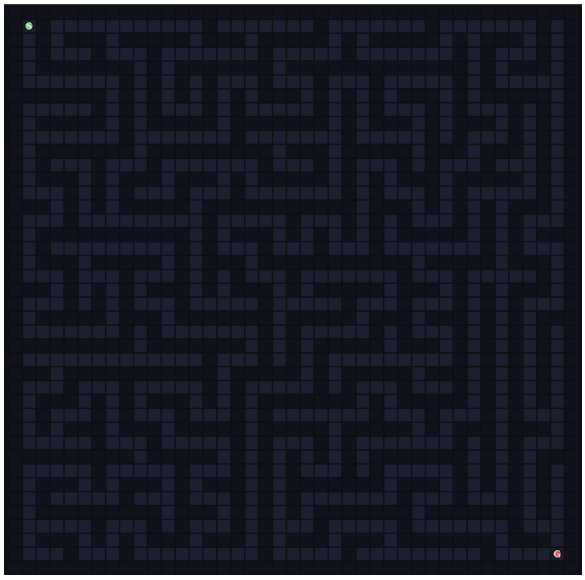

Figure 3.2: Randomly Generated Maze (Recursive Backtracking)

The control panel provides algorithm selection (BFS / DFS / Dijkstra / A*), maze size and generator toggles, the speed slider, and action buttons.

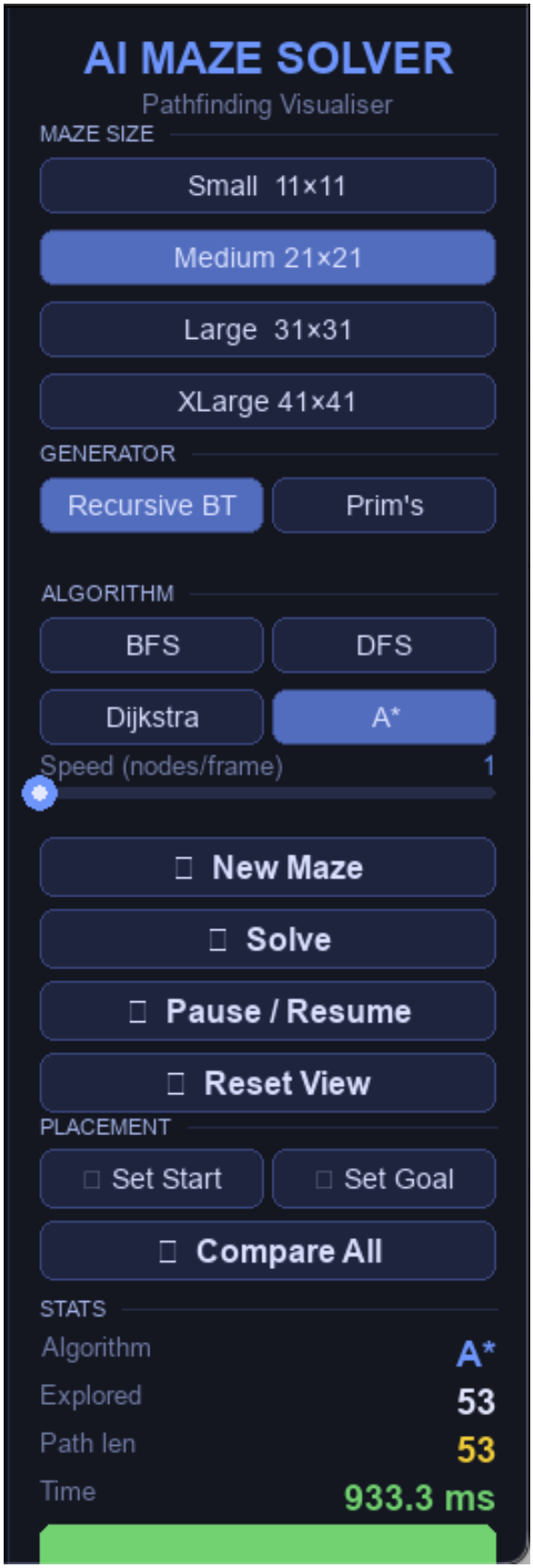

Figure 3.3: Sidebar Control Panel

The statistics panel displays real-time performance metrics after solving.

Figure 3.4: Statistics Display Panel Showing Performance Metrics

The next figure captures mid-execution, showing explored cells (blue gradient) spreading outward from the start.

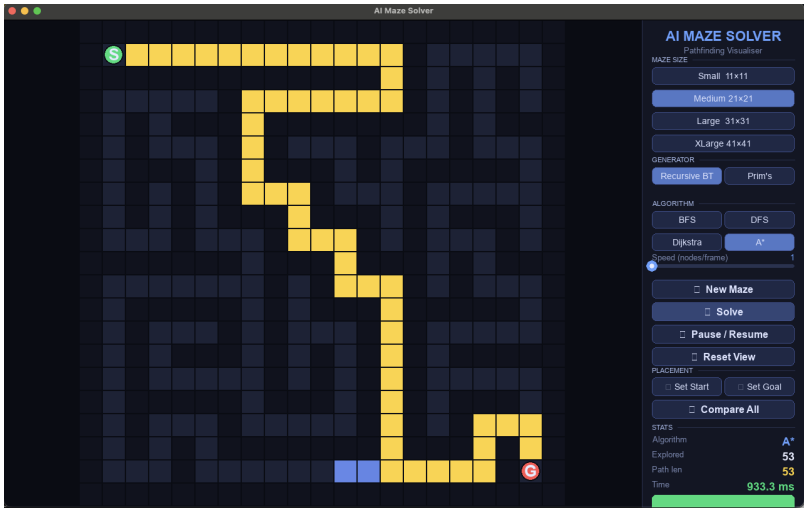

Figure 3.5: Algorithm Actively Exploring the Maze

The final figure shows the completed solve with the optimal solution path highlighted in gold.

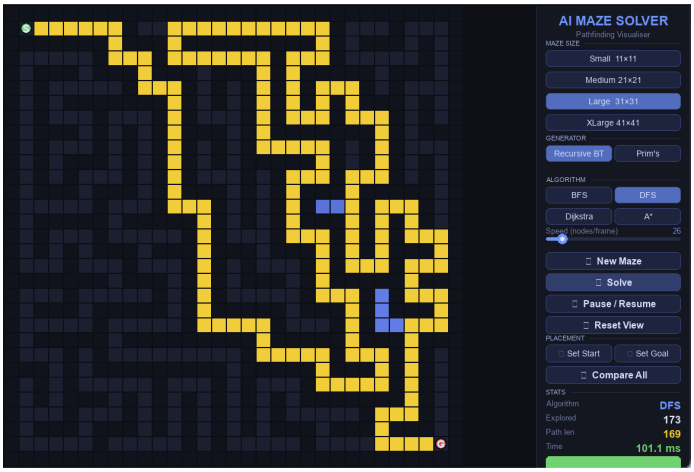

Figure 3.6: Solved Maze with Shortest Path Highlighted in Gold

3.4 Overall Discussion

The results confirm several well-known theoretical properties of these algorithms and demonstrate their practical manifestation in a visual environment.

BFS and Dijkstra explore the search space uniformly and guarantee optimality at the cost of high node counts. Their traversal patterns are visually distinctive: BFS produces a near-circular frontier that expands evenly in all directions, while Dijkstra shows an identical pattern for unit-weight mazes.

DFS dives aggressively along a single corridor before backtracking, often reaching the goal with fewer frame-ticks but via a much longer route. Its exploration pattern is characteristically irregular — long tendrils rather than a smooth frontier.

A* combines the guarantee of optimality with the efficiency of heuristic guidance. Its exploration is visibly directional: the frontier stretches toward the goal rather than expanding uniformly. This makes A* the preferred algorithm whenever both correctness and speed matter.

The generator-based animation architecture proved highly effective. Decoupling the solver from the renderer allowed the speed slider to vary from single-node steps to 200 nodes per frame without any changes to the algorithm implementations — a clean separation of concerns (EA1).

Chapter 4

Conclusion

4.1 Discussion

The AI Maze Game Solver successfully achieves its design objectives: four pathfinding algorithms are implemented as animation-ready generators, two procedural generation methods produce varied and solvable mazes, and a Pygame-based visualiser brings all components together in an interactive educational tool.

The project demonstrates that theoretical algorithm properties — BFS optimality, DFS speed–quality trade-off, Dijkstra's cost accounting, A*'s heuristic focus — are faithfully reproduced in a real interactive system and are immediately apparent to an observer watching the animations. The Compare All mode provides empirical evidence of these differences in a single click (WP3).

The generator architecture is the project's most significant software engineering contribution. By reducing each algorithm to a coroutine that yields one exploration step at a time, the system cleanly separates the "what to compute" concern (solver module) from the "how fast to render" concern (visualiser module), yielding code that is both more readable and more maintainable than a thread-based alternative (EA1).

4.2 Limitations

- The maze is strictly two-dimensional and 4-connected; diagonal movement and 3D environments are not supported.
- All cells have unit cost; weighted terrain (mud, water) is not modelled, which limits Dijkstra's distinguishability from BFS.
- Maze size is capped at 41×41 in the UI; very large grids (100×100+) may cause visible slowdowns at low speed settings.
- The Recursive Backtracking generator uses Python's call stack, requiring a recursion limit increase (sys.setrecursionlimit(10000)) for larger grids.
- The application is desktop-only (Pygame); no web or mobile deployment is available.

4.3 Scope of Future Work

- **Weighted terrain:** Introduce cell cost values so Dijkstra and A* can demonstrate advantages over BFS on non-uniform grids (WP2).
- **Bidirectional search:** Implement bidirectional BFS and bidirectional A* to further reduce explored node counts (WP1, WP3).
- **Q-Learning agent:** Add a reinforcement-learning agent that learns to navigate the maze through trial-and-reward, contrasting model-free AI with classical search (EA3).
- **Multiple waypoints:** Extend the problem to visit intermediate checkpoints before reaching the goal (WP2).
- **Wall editor:** Allow users to draw and erase walls with the mouse to create custom maze layouts (EA2).
- **Web deployment:** Port the visualiser to Pygame-CE/WebAssembly or a JavaScript framework for browser-based access (EA2, EA3).
- **Export feature:** Save maze layouts and solution images as PNG files for use in reports or presentations.

References

[1] Russell, S. & Norvig, P. *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2021.

[2] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. *Introduction to Algorithms*, 3rd ed. MIT Press, 2009.

[3] Sedgewick, R. & Wayne, K. *Algorithms*, 4th ed. Addison-Wesley, 2011.

[4] Pygame Community. *Pygame Documentation*. https://www.pygame.org/docs/

[5] Amit Patel, Red Blob Games. *Introduction to the A* Algorithm*. https://www.redblobgames.com/pathfinding/a-star/introduction.html

[6] GeeksforGeeks. *Breadth First Search or BFS for a Graph*. https://www.geeksforgeeks.org/breadth-first-search-or-bfs-for-a-graph/

[7] Wikipedia contributors. *Maze generation algorithm*. https://en.wikipedia.org/wiki/Maze_generation_algorithm

Chapter 5

Appendix: Source Code

5.1 maze.py — Maze Generation

```
import random, sys

class Maze:
    def __init__(self, rows: int, cols: int):
        self.rows = rows if rows % 2 == 1 else rows + 1
        self.cols = cols if cols % 2 == 1 else cols + 1
        self.grid = [[1] * self.cols for _ in range(self.rows)]
        self.start = (1, 1)
        self.goal  = (self.rows - 2, self.cols - 2)

    # --- Recursive Backtracking ---
    def generate_recursive_backtracking(self):
        self.grid = [[1] * self.cols for _ in range(self.rows)]
        visited   = [[False] * self.cols for _ in range(self.
           rows)]
        def carve(r, c):
            visited[r][c] = True
            self.grid[r][c] = 0
            dirs = [(0,2),(2,0),(0,-2),(-2,0)]
            random.shuffle(dirs)
            for dr, dc in dirs:
                nr, nc = r+dr, c+dc
                if (0 < nr < self.rows-1 and 0 < nc < self.cols
                   -1
                        and not visited[nr][nc]):
                    self.grid[r+dr//2][c+dc//2] = 0
                    carve(nr, nc)
        sys.setrecursionlimit(10_000)
        carve(1, 1)
        self._place_markers()

    # --- Prim's Algorithm ---
    def generate_prims(self):
```

```
        self.grid = [[1] * self.cols for _ in range(self.rows)]
        self.grid[1][1] = 0
        frontier = []
        def add_frontiers(r, c):
            for dr, dc in [(0,2),(2,0),(0,-2),(-2,0)]:
                nr, nc = r+dr, c+dc
                if (0 < nr < self.rows-1 and 0 < nc < self.cols
                   -1
                        and self.grid[nr][nc] == 1):
                    frontier.append((nr, nc, r, c))
        add_frontiers(1, 1)
        while frontier:
            idx = random.randrange(len(frontier))
            nr, nc, pr, pc = frontier.pop(idx)
            if self.grid[nr][nc] == 1:
                self.grid[nr][nc] = 0
                self.grid[(nr+pr)//2][(nc+pc)//2] = 0
                add_frontiers(nr, nc)
        self._place_markers()

    def _place_markers(self):
        self.grid[self.start[0]][self.start[1]] = 2
        self.grid[self.goal[0]][self.goal[1]]   = 3

    def is_walkable(self, r, c):
        return self.grid[r][c] != 1

    def neighbors(self, r, c):
        for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
            nr, nc = r+dr, c+dc
            if 0 <= nr < self.rows and 0 <= nc < self.cols:
                if self.grid[nr][nc] != 1:
                    yield nr, nc
```

Listing 5.1: Maze data model and generation algorithms

5.2 solver.py — Pathfinding Algorithms

```
import heapq
from collections import deque
from maze import Maze

def _reconstruct_path(came_from, end):
    path, node = [], end
    while node is not None:
        path.append(node)
        node = came_from.get(node)
    path.reverse()
    return path
```

```python
def bfs(maze: Maze):
    s, t = maze.start, maze.goal
    q = deque([s])
    cf = {s: None}
    while q:
        v = q.popleft()
        yield ("explore", [v])
        if v == t:
            yield ("path", _reconstruct_path(cf, t)); return
        for u in maze.neighbors(*v):
            if u not in cf:
                cf[u] = v; q.append(u)
    yield ("path", [])

def dfs(maze: Maze):
    s, t = maze.start, maze.goal
    stack = [s]
    cf = {s: None}
    while stack:
        v = stack.pop()
        yield ("explore", [v])
        if v == t:
            yield ("path", _reconstruct_path(cf, t)); return
        for u in maze.neighbors(*v):
            if u not in cf:
                cf[u] = v; stack.append(u)
    yield ("path", [])

def dijkstra(maze: Maze):
    s, t = maze.start, maze.goal
    dist = {s: 0}
    cf   = {s: None}
    heap = [(0, s)]
    while heap:
        d, v = heapq.heappop(heap)
        yield ("explore", [v])
        if v == t:
            yield ("path", _reconstruct_path(cf, t)); return
        if d > dist.get(v, float("inf")):
            continue
        for u in maze.neighbors(*v):
            nd = d + 1
            if nd < dist.get(u, float("inf")):
                dist[u] = nd; cf[u] = v
                heapq.heappush(heap, (nd, u))
    yield ("path", [])

def _manhattan(a, b):
    return abs(a[0]-b[0]) + abs(a[1]-b[1])
```

```python
def astar(maze: Maze):
    s, t   = maze.start, maze.goal
    g      = {s: 0}
    cf     = {s: None}
    heap   = [(_manhattan(s,t), 0, s)]
    closed = set()
    while heap:
        _, gc, v = heapq.heappop(heap)
        if v in closed: continue
        closed.add(v)
        yield ("explore", [v])
        if v == t:
            yield ("path", _reconstruct_path(cf, t)); return
        for u in maze.neighbors(*v):
            if u in closed: continue
            ng = gc + 1
            if ng < g.get(u, float("inf")):
                g[u] = ng; cf[u] = v
                heapq.heappush(heap, (ng + _manhattan(u,t), ng,
                    u))
    yield ("path", [])

ALGORITHMS = {"BFS": bfs, "DFS": dfs, "Dijkstra": dijkstra, "A*
    ": astar}
```

Listing 5.2: BFS, DFS, Dijkstra, and A* as generator functions

5.3 visualizer.py — Pygame-based visualiser for the AI Maze Solver Layout

```python
import math
import time
from typing import List, Optional, Tuple

import pygame

from maze import Maze

# Palette      deep dark theme with vibrant accents
BG           = (10,  12,  20)
PANEL_BG     = (18,  22,  36)
PANEL_BORDER= (40,  50,  80)

WALL         = (15,  18,  30)
PATH         = (28,  34,  55)
START_CLR    = (0,  220, 120)
GOAL_CLR     = (255, 80,  80)
```

```
EXPLORED     = (60,  90, 180)
FRONTIER     = (100, 140, 240)
FINAL_PATH   = (255, 210,  50)

TEXT_PRIMARY   = (230, 235, 255)
TEXT_SECONDARY = (120, 135, 175)
TEXT_SECTION   = (168, 188, 235)
ACCENT         = (100, 160, 255)
ACCENT2        = (160, 100, 255)

BTN_IDLE     = (30,  40,  70)
BTN_HOVER    = (50,  70, 120)
BTN_ACTIVE   = (80, 120, 200)
BTN_TEXT     = (220, 228, 255)
BTN_BORDER   = (60,  80, 140)

SLIDER_RAIL = (40,  50,  80)
SLIDER_KNOB = (100, 160, 255)

# Constants
SIDEBAR_W    = 260
MIN_CELL     = 8
FPS          = 60

MAZE_SIZES   = [("Small  11 11 ", 11), ("Medium 21 21 ", 21),
                ("Large  31 31 ", 31), ("XLarge 41 41 ", 41)]
GEN_MODES    = ["Recursive BT", "Prim's"]
ALGO_NAMES   = list(ALGORITHMS.keys())    # BFS, DFS, Dijkstra, A
    *

SPEED_MIN, SPEED_MAX = 1, 200             # nodes animated per
    frame


# Small UI helpers


def lerp_color(a, b, t):
    t = max(0.0, min(1.0, t))
    return tuple(int(a[i] + (b[i] - a[i]) * t) for i in range
        (3))


def draw_rounded_rect(surf, color, rect, radius=8, border=0,
    border_color=None):
    pygame.draw.rect(surf, color, rect, border_radius=radius)
    if border and border_color:
        pygame.draw.rect(surf, border_color, rect, border,
            border_radius=radius)
```

```python
def draw_text(surf, text, font, color, x, y, anchor="topleft"):
    img = font.render(text, True, color)
    r = img.get_rect(**{anchor: (x, y)})
    surf.blit(img, r)
    return r



# Button

class Button:
    def __init__(self, rect, label, tag=None, toggle=False,
       active=False):
        self.rect   = pygame.Rect(rect)
        self.label  = label
        self.tag    = tag
        self.toggle = toggle
        self.active = active
        self._hover = False
        self._t     = 0.0          # animation interpolant

    def handle_event(self, event):
        clicked = False
        if event.type == pygame.MOUSEMOTION:
            self._hover = self.rect.collidepoint(event.pos)
        if event.type == pygame.MOUSEBUTTONDOWN and event.
           button == 1:
            if self.rect.collidepoint(event.pos):
                if self.toggle:
                    self.active = not self.active
                clicked = True
        return clicked

    def draw(self, surf, font):
        target_t = 1.0 if (self._hover or self.active) else 0.0
        self._t  = self._t + (target_t - self._t) * 0.25
        base  = BTN_ACTIVE if self.active else BTN_IDLE
        hover = BTN_HOVER  if not self.active else BTN_ACTIVE
        color = lerp_color(base, hover, self._t)
        draw_rounded_rect(surf, color, self.rect, 7, 1,
           BTN_BORDER)
        cx, cy = self.rect.center
        draw_text(surf, self.label, font, BTN_TEXT, cx, cy, "
           center")




class Slider:
    def __init__(self, x, y, w, lo, hi, value, label):
        self.rect  = pygame.Rect(x, y, w, 6)
```

```
        self.lo, self.hi = lo, hi
        self.value = value
        self.label = label
        self._drag = False

    @property
    def knob_x(self):
        t = (self.value - self.lo) / (self.hi - self.lo)
        return int(self.rect.x + t * self.rect.w)

    def handle_event(self, event):
        if event.type == pygame.MOUSEBUTTONDOWN and event.
           button == 1:
            kx = self.knob_x
            ky = self.rect.centery
            if math.hypot(event.pos[0] - kx, event.pos[1] - ky)
                < 14:
                self._drag = True
        if event.type == pygame.MOUSEBUTTONUP:
            self._drag = False
        if event.type == pygame.MOUSEMOTION and self._drag:
            t = (event.pos[0] - self.rect.x) / self.rect.w
            self.value = int(self.lo + max(0, min(1, t)) * (
               self.hi - self.lo))

    def draw(self, surf, font_sm, font_xs):
        # Rail
        pygame.draw.rect(surf, SLIDER_RAIL, self.rect,
           border_radius=3)
        # Filled portion
        filled = pygame.Rect(self.rect.x, self.rect.y,
                             self.knob_x - self.rect.x, self.
                                rect.h)
        pygame.draw.rect(surf, ACCENT, filled, border_radius=3)
        # Knob
        pygame.draw.circle(surf, SLIDER_KNOB, (self.knob_x,
           self.rect.centery), 9)
        pygame.draw.circle(surf, TEXT_PRIMARY, (self.knob_x,
           self.rect.centery), 4)
        # Labels
        draw_text(surf, self.label, font_sm, TEXT_SECONDARY,
                  self.rect.x, self.rect.y - 18)
        draw_text(surf, str(self.value), font_sm, ACCENT,
                  self.rect.right, self.rect.y - 18, "topright"
                     )


#
   ---------------------------------------------------------------------------

# Main Visualiser class
```

```
#
    ----------------------
```

```
    def _init_state(self):
        self.maze:          Optional[Maze]        = None
        self.explored:      List[Tuple[int,int]] = []
        self.final_path:    List[Tuple[int,int]] = []
        self.generator                            = None
        self.solving        = False
        self.solved         = False
        self.paused         = False

        # Perf stats
        self.nodes_explored = 0
        self.path_length    = 0
        self.elapsed_ms     = 0.0
        self._solve_start   = 0.0

        # Selection state
        self.sel_size_idx   = 1       # Medium 21 21
        self.sel_gen_idx    = 0       # Recursive BT
        self.sel_algo_idx   = 3       # A*

        # Comparison results
        self.cmp_results: dict = {}

        # Placement mode: None | "start" | "goal"
        self.placement_mode: Optional[str] = None

    #         UI widgets
    def _build_ui(self):
        sx = self.screen_w - SIDEBAR_W + 16
        w  = SIDEBAR_W - 32

        #         Maze size buttons
        self.btn_sizes = []
        for i, (label, _) in enumerate(MAZE_SIZES):
            r = pygame.Rect(sx, 76 + i * 36, w, 28)
            b = Button(r, label, tag=i)
            b.active = (i == self.sel_size_idx)
            self.btn_sizes.append(b)

        #         Generation buttons
        self.btn_gens = []
        for i, label in enumerate(GEN_MODES):
            r = pygame.Rect(sx + i * (w // 2 + 2), 236, w // 2
               - 2, 28)
            b = Button(r, label, tag=i)
            b.active = (i == self.sel_gen_idx)
            self.btn_gens.append(b)

        #         Algorithm buttons
        self.btn_algos = []
```

```
        for i, name in enumerate(ALGO_NAMES):
            col = i % 2
            row = i // 2
            bw = w // 2 - 2
            r  = pygame.Rect(sx + col * (bw + 4), 306 + row *
               36, bw, 28)
            b  = Button(r, name, tag=i)
            b.active = (i == self.sel_algo_idx)
            self.btn_algos.append(b)

        #         Speed slider
        self.slider_speed = Slider(sx, 391, w, SPEED_MIN,
           SPEED_MAX, 30, "Speed (nodes/frame)")

        #         Action buttons
        self.btn_generate = Button(pygame.Rect(sx, 416, w, 30),
            "      New Maze")
        self.btn_solve    = Button(pygame.Rect(sx, 452, w, 30),
            "      Solve")
        self.btn_pause    = Button(pygame.Rect(sx, 488, w, 30),
            "      Pause / Resume")
        self.btn_reset    = Button(pygame.Rect(sx, 524, w, 30),
            "      Reset View")

        #         Placement buttons (half-width pair)
        hw = w // 2 - 2
        self.btn_set_start = Button(pygame.Rect(sx,       572,
           hw, 30), "       Set Start")
        self.btn_set_goal  = Button(pygame.Rect(sx + hw + 4,
           572, hw, 30), "       Set Goal")

        # Used to position section labels so they don't collide
            with nearby buttons.
        self.y_placement_label = self.btn_set_start.rect.y - (
           self.font_xs.get_height() + 4)

        self.btn_compare  = Button(pygame.Rect(sx, 608, w, 30),
            "      Compare All")

    def _rebuild_ui_positions(self):
        """Rebuild UI after window resize."""
        self._build_ui()
        # Restore selections
        for i, b in enumerate(self.btn_sizes):
            b.active = (i == self.sel_size_idx)
        for i, b in enumerate(self.btn_gens):
            b.active = (i == self.sel_gen_idx)
        for i, b in enumerate(self.btn_algos):
            b.active = (i == self.sel_algo_idx)
        self.slider_speed.value = min(SPEED_MAX, max(SPEED_MIN,
```

```
                                        self.slider_speed.value)
                                            )

    #        Maze / solver control functions

    def _new_maze(self):
        self._stop_solving()
        rows = cols = MAZE_SIZES[self.sel_size_idx][1]
        self.maze = Maze(rows, cols)
        if self.sel_gen_idx == 0:
            self.maze.generate_recursive_backtracking()
        else:
            self.maze.generate_prims()
        self.explored   = []
        self.final_path = []
        self.cmp_results = {}
        self._reset_stats()

    def _start_solve(self):
        if self.maze is None:
            return
        self._stop_solving()
        self.maze.reset_markers()
        self.explored   = []
        self.final_path = []
        self._reset_stats()
        algo_fn         = ALGORITHMS[ALGO_NAMES[self.
           sel_algo_idx]]
        self.generator  = algo_fn(self.maze)
        self.solving    = True
        self.solved     = False
        self.paused     = False
        self._solve_start = time.perf_counter()

    def _stop_solving(self):
        self.solving   = False
        self.generator = None

    def _reset_stats(self):
        self.nodes_explored = 0
        self.path_length    = 0
        self.elapsed_ms     = 0.0

    def _toggle_pause(self):
        if self.solving:
            self.paused = not self.paused

    def _reset_view(self):
        self._stop_solving()
        self.explored    = []
        self.final_path  = []
```

```
        self.cmp_results = {}
        self.placement_mode = None
        self.maze.reset_markers()
        self._reset_stats()

    def _pixel_to_cell(self, mx, my):
        """Convert screen pixel (mx, my) to maze (row, col), or
            None if outside."""
        if self.maze is None:
            return None
        cs     = self._cell_size()
        maze_w = self.screen_w - SIDEBAR_W
        off_x  = (maze_w - self.maze.cols * cs) // 2
        off_y  = (self.screen_h - self.maze.rows * cs) // 2
        c = (mx - off_x) // cs
        r = (my - off_y) // cs
        if 0 <= r < self.maze.rows and 0 <= c < self.maze.cols:
            return (r, c)
        return None

    #        Step solver generator to animate it
    def _tick_solver(self):
        if not self.solving or self.paused or self.generator is
            None:
            return
        steps = self.slider_speed.value
        for _ in range(steps):
            try:
                kind, cells = next(self.generator)
            except StopIteration:
                self.solving = False
                self.solved  = True
                self.elapsed_ms = (time.perf_counter() - self.
                    _solve_start) * 1000
                return

            if kind == "explore":
                for cell in cells:
                    if cell not in (self.maze.start, self.maze.
                        goal):
                         self.explored.append(cell)
                self.nodes_explored = len(self.explored)
            elif kind == "path":
                self.final_path  = cells
                self.path_length = len(cells)
                self.solving     = False
                self.solved      = True
                self.elapsed_ms  = (time.perf_counter() - self.
                    _solve_start) * 1000
                return
```

```python
    #         Comparison (instant, no animation)

    def _run_compare(self):
        if self.maze is None:
            return
        self._stop_solving()
        self.cmp_results = {}
        for name, fn in ALGORITHMS.items():
            self.maze.reset_markers()
            gen   = fn(self.maze)
            count = 0
            path  = []
            t0    = time.perf_counter()
            for kind, cells in gen:
                if kind == "explore":
                    count += len(cells)
                elif kind == "path":
                    path = cells
            elapsed = (time.perf_counter() - t0) * 1000
            self.cmp_results[name] = {
                "nodes":   count,
                "path":    len(path),
                "time_ms": elapsed,
            }
        self.explored   = []
        self.final_path = []
        self.maze.reset_markers()

    #         Drawing functions

    def _cell_size(self):
        maze_w = self.screen_w - SIDEBAR_W
        maze_h = self.screen_h
        if self.maze is None:
            return MIN_CELL
        cs = min(maze_w // self.maze.cols, maze_h // self.maze.
           rows)
        return max(MIN_CELL, cs)

    def _draw_maze(self):
        if self.maze is None:
            return
        cs    = self._cell_size()
        maze_w = self.screen_w - SIDEBAR_W
        maze_h = self.screen_h
        off_x = (maze_w - self.maze.cols * cs) // 2
        off_y = (maze_h - self.maze.rows * cs) // 2

        explored_set   = set(self.explored)
        final_path_set = set(self.final_path)
```

```
        for r in range(self.maze.rows):
            for c in range(self.maze.cols):
                cell = (r, c)
                v    = self.maze.grid[r][c]
                rect = pygame.Rect(off_x + c * cs, off_y + r *
                   cs, cs, cs)

                if v == 1:                          # wall
                    color = WALL
                elif v == 2:                        # start (marker
                    drawn later)
                    color = PATH
                elif v == 3:                        # goal (marker
                   drawn later)
                    color = PATH
                elif cell in final_path_set:        # final path
                    color = FINAL_PATH
                elif cell in explored_set:          # explored
                    # gradient: darker early, brighter late
                    idx = self.explored.index(cell)
                    t   = idx / max(1, len(self.explored) - 1)
                    color = lerp_color(EXPLORED, FRONTIER, t)
                else:
                    color = PATH

                pygame.draw.rect(self.screen, color, rect)
                if cs >= 10:
                    pygame.draw.rect(self.screen, BG, rect, 1)

        # Start / goal markers (circles constrained to 1 cell)
        self._draw_marker_circle(
            off_x + self.maze.start[1] * cs + cs // 2,
            off_y + self.maze.start[0] * cs + cs // 2,
            START_CLR,
            cs,
        )
        self._draw_marker_circle(
            off_x + self.maze.goal[1] * cs + cs // 2,
            off_y + self.maze.goal[0] * cs + cs // 2,
            GOAL_CLR,
            cs,
        )

        # Labels on start / goal
        self._draw_cell_label(
            "S",
            off_x + self.maze.start[1] * cs + cs // 2,
            off_y + self.maze.start[0] * cs + cs // 2,
            cs,
            TEXT_PRIMARY,
        )
```

```
        self._draw_cell_label(
            "G",
            off_x + self.maze.goal[1] * cs + cs // 2,
            off_y + self.maze.goal[0] * cs + cs // 2,
            cs,
            TEXT_PRIMARY,
        )

        #         Placement mode overlay hint
        if self.placement_mode:
            clr  = START_CLR if self.placement_mode == "start"
               else GOAL_CLR
            icon = "      " if self.placement_mode == "start"
               else "      "
            msg  = f"{icon}  Click any open cell to place {'
               START' if self.placement_mode == 'start' else '
               GOAL'}   [Esc] to cancel"
            hint = self.font_label.render(msg, True, clr)
            hw, hh = hint.get_size()
            pad = 10
            bg_rect = pygame.Rect((maze_w - hw) // 2 - pad, 14,
                hw + pad * 2, hh + pad)
            s = pygame.Surface((bg_rect.w, bg_rect.h), pygame.
               SRCALPHA)
            s.fill((10, 12, 20, 200))
            self.screen.blit(s, bg_rect.topleft)
            pygame.draw.rect(self.screen, clr, bg_rect, 2,
               border_radius=8)
            self.screen.blit(hint, ((maze_w - hw) // 2, 14 +
               pad // 2))

    def _draw_marker_circle(self, cx: int, cy: int, color, cs:
       int):
        # Fit the marker within a single cell ("1 1  board size
           ").
        radius = max(3, cs // 2 - 2)
        pygame.draw.circle(self.screen, color, (cx, cy), radius
           )
        pygame.draw.circle(self.screen, BG, (cx, cy), radius,
           2)

    def _cell_label_font(self, cs: int) -> pygame.font.Font:
        # Cache a font sized to the current cell size (
           recreated on resize).
        size = max(12, int(cs * 0.72))
        cache = getattr(self, "_cell_font_cache", None)
        if cache is None:
            cache = {}
            self._cell_font_cache = cache
        if size not in cache:
```

```
            cache[size] = pygame.font.SysFont("DejaVu Sans",
                size, bold=True)
        return cache[size]

    def _draw_cell_label(self, text: str, cx: int, cy: int, cs:
        int, color):
        # Only draw labels when cells are big enough to be
            readable.
        if cs < 12:
            return
        font = self._cell_label_font(cs)
        img = font.render(text, True, color)
        r = img.get_rect(center=(cx, cy))
        self.screen.blit(img, r)

    def _draw_sidebar(self):
        sx = self.screen_w - SIDEBAR_W
        sidebar_rect = pygame.Rect(sx, 0, SIDEBAR_W, self.
            screen_h)
        draw_rounded_rect(self.screen, PANEL_BG, sidebar_rect,
            0, 1, PANEL_BORDER)

        #         Title
        draw_text(self.screen, "AI MAZE SOLVER",
                  self.font_title, ACCENT, sx + SIDEBAR_W // 2,
                      14, "midtop")
        draw_text(self.screen, "Pathfinding Visualiser",
                  self.font_small, TEXT_SECONDARY, sx +
                     SIDEBAR_W // 2, 42, "midtop")

        #         Section: Maze Size
        self._section_label("MAZE SIZE", sx + 16, 58)
        for b in self.btn_sizes:
            b.draw(self.screen, self.font_body)

        #         Section: Generator
        self._section_label("GENERATOR", sx + 16, 218)
        for b in self.btn_gens:
            b.draw(self.screen, self.font_body)

        #         Section: Algorithm
        self._section_label("ALGORITHM", sx + 16, 288)
        for b in self.btn_algos:
            b.draw(self.screen, self.font_body)

        #         Speed slider
        self.slider_speed.draw(self.screen, self.font_small,
            self.font_xs)

        #         Action buttons
        self.btn_generate.draw(self.screen, self.font_label)
```

```
        self.btn_solve.draw(self.screen, self.font_label)
        self.btn_pause.draw(self.screen, self.font_label)
        self.btn_reset.draw(self.screen, self.font_label)

        #         Placement section
        self._section_label("PLACEMENT", sx + 16, self.
           y_placement_label)
        # Highlight active placement button
        self.btn_set_start.active = (self.placement_mode == "
           start")
        self.btn_set_goal.active  = (self.placement_mode == "
           goal")
        self.btn_set_start.draw(self.screen, self.font_body)
        self.btn_set_goal.draw(self.screen, self.font_body)

        self.btn_compare.draw(self.screen, self.font_label)

        #         Stats panel
        self._draw_stats(sx + 16, 646)

    def _section_label(self, text, x, y):
        label_rect = draw_text(self.screen, text, self.font_xs,
            TEXT_SECTION, x, y)
        line_y = label_rect.centery
        line_start = label_rect.right + 8
        if line_start < self.screen_w - 16:
            pygame.draw.line(self.screen, PANEL_BORDER,
                             (line_start, line_y), (self.
                                screen_w - 16, line_y), 1)

    def _draw_stats(self, x, y):
        self._section_label("STATS", x, y)
        y += 14

        algo_name = ALGO_NAMES[self.sel_algo_idx]
        items = [
            ("Algorithm",  algo_name,                ACCENT),
            ("Explored",   str(self.nodes_explored),
               TEXT_PRIMARY),
            ("Path len",   str(self.path_length),    FINAL_PATH
                if self.path_length else TEXT_PRIMARY),
            ("Time",       f"{self.elapsed_ms:.1f} ms",
               START_CLR),
        ]
        for label, value, clr in items:
            draw_text(self.screen, label, self.font_small,
               TEXT_SECONDARY, x, y + 4)
            draw_text(self.screen, value, self.font_stat, clr,
                      self.screen_w - 16, y + 4, "topright")
            y += 24
```

```
        # Status badge
        if self.solving:
            status, sc = ("SOLVING ", ACCENT)
        elif self.solved:
            status, sc = ("SOLVED    ", START_CLR) if self.
               path_length else ("NO PATH    ", GOAL_CLR)
        else:
            status, sc = ("READY", TEXT_SECONDARY)

        badge = pygame.Rect(x, y + 4, SIDEBAR_W - 32, 24)
        draw_rounded_rect(self.screen, (*sc[:3], 30), badge, 6)
        draw_text(self.screen, status, self.font_label, sc,
                  badge.centerx, badge.centery, "center")

    def _draw_comparison(self):
        if not self.cmp_results:
            return
        panel_w = 520
        panel_h = 260
        px = (self.screen_w - SIDEBAR_W - panel_w) // 2
        py = (self.screen_h - panel_h) // 2
        panel = pygame.Rect(px, py, panel_w, panel_h)
        draw_rounded_rect(self.screen, PANEL_BG, panel, 12, 2,
           PANEL_BORDER)

        draw_text(self.screen, "Algorithm Comparison", self.
           font_title, ACCENT,
                  panel.centerx, py + 14, "midtop")

        headers = ["Algorithm", "Nodes Explored", "Path Length"
           , "Time (ms)"]
        col_xs  = [px + 16, px + 150, px + 300, px + 410]
        hy = py + 50
        for h, cx in zip(headers, col_xs):
            draw_text(self.screen, h, self.font_small,
               TEXT_SECONDARY, cx, hy)

        pygame.draw.line(self.screen, PANEL_BORDER,
                         (px + 16, hy + 18), (panel.right - 16,
                             hy + 18), 1)

        ry = hy + 28
        colors = [ACCENT, ACCENT2, START_CLR, FINAL_PATH]
        best_nodes = min(v["nodes"] for v in self.cmp_results.
           values())
        best_path  = min((v["path"] for v in self.cmp_results.
           values() if v["path"] > 0), default=0)
        best_time  = min(v["time_ms"] for v in self.cmp_results
           .values())
```

```
        for i, (name, data) in enumerate(self.cmp_results.items
            ()):
            clr  = colors[i % len(colors)]
            draw_text(self.screen, name, self.font_label, clr,
                col_xs[0], ry)

            nc = START_CLR if data["nodes"] == best_nodes else
                TEXT_PRIMARY
            draw_text(self.screen, str(data["nodes"]), self.
                font_body, nc, col_xs[1], ry)

            pl = data["path"]
            pc = START_CLR if (pl > 0 and pl == best_path) else
                 (GOAL_CLR if pl == 0 else TEXT_PRIMARY)
            draw_text(self.screen, str(pl) if pl else "N/A",
                self.font_body, pc, col_xs[2], ry)

            tc = START_CLR if data["time_ms"] == best_time else
                 TEXT_PRIMARY
            draw_text(self.screen, f"{data['time_ms']:.2f}",
                self.font_body, tc, col_xs[3], ry)
            ry += 30

        draw_text(self.screen, "    = best  |  click anywhere
            to dismiss",
                  self.font_xs, TEXT_SECONDARY, panel.centerx,
                     panel.bottom - 16, "midbottom")

    #        Event handling

    def _handle_events(self):
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                return False
            if event.type == pygame.VIDEORESIZE:
                self.screen_w, self.screen_h = event.w, event.h
                self.screen = pygame.display.set_mode(
                    (self.screen_w, self.screen_h), pygame.
                       RESIZABLE)
                self._rebuild_ui_positions()

            # Dismiss comparison panel
            if self.cmp_results and event.type == pygame.
                MOUSEBUTTONDOWN:
                self.cmp_results = {}
                continue

            # Cancel placement mode with Escape
            if event.type == pygame.KEYDOWN and event.key ==
                pygame.K_ESCAPE:
                self.placement_mode = None
```

```
        # Canvas click     place start or goal
        if (event.type == pygame.MOUSEBUTTONDOWN
                and event.button == 1
                and self.placement_mode
                and self.maze):
            mx, my = event.pos
            if mx < self.screen_w - SIDEBAR_W:   # clicked
               on canvas
                cell = self._pixel_to_cell(mx, my)
                if cell:
                    if self.placement_mode == "start":
                        if self.maze.set_start(*cell):
                            self._reset_view()
                            self.placement_mode = None
                    else:
                        if self.maze.set_goal(*cell):
                            self._reset_view()
                            self.placement_mode = None
                continue   # don't let other handlers
                   process this click

        # Size buttons
        for i, b in enumerate(self.btn_sizes):
            if b.handle_event(event):
                self.sel_size_idx = i
                for other in self.btn_sizes:
                    other.active = False
                b.active = True
                self._new_maze()

        # Generator buttons
        for i, b in enumerate(self.btn_gens):
            if b.handle_event(event):
                self.sel_gen_idx = i
                for other in self.btn_gens:
                    other.active = False
                b.active = True

        # Algorithm buttons
        for i, b in enumerate(self.btn_algos):
            if b.handle_event(event):
                self.sel_algo_idx = i
                for other in self.btn_algos:
                    other.active = False
                b.active = True

        # Slider
        self.slider_speed.handle_event(event)

        # Action buttons
```

```
            if self.btn_generate.handle_event(event):
                self._new_maze()
            if self.btn_solve.handle_event(event):
                self._start_solve()
            if self.btn_pause.handle_event(event):
                self._toggle_pause()
            if self.btn_reset.handle_event(event):
                self._reset_view()
            if self.btn_compare.handle_event(event):
                self._run_compare()

            # Placement mode toggle
            if self.btn_set_start.handle_event(event):
                self.placement_mode = None if self.
                   placement_mode == "start" else "start"
            if self.btn_set_goal.handle_event(event):
                self.placement_mode = None if self.
                   placement_mode == "goal" else "goal"

            # Keyboard shortcuts
            if event.type == pygame.KEYDOWN:
                if event.key == pygame.K_SPACE:
                    self._toggle_pause()
                if event.key == pygame.K_RETURN:
                    self._start_solve()
                if event.key == pygame.K_n:
                    self._new_maze()
                if event.key == pygame.K_r:
                    self._reset_view()

        return True

    #        Animated intro gradient title on canvas
    def _draw_canvas_bg(self):
        self.screen.fill(BG)

    #        Main loop
    def run(self):
        running = True
        while running:
            running = self._handle_events()
            self._tick_solver()

            self._draw_canvas_bg()
            self._draw_maze()
            self._draw_sidebar()
            if self.cmp_results:
                self._draw_comparison()

            pygame.display.flip()
            self.clock.tick(FPS)
```

```
        pygame.quit()
```

Listing 5.3: Pygame-based visualiser for the AI Maze Solver Layout